

Installing CellProfiler with the RunCellpose module

Pixi as the recommended method and Docker as an alternative

Table of contents

1	Option 1 — Installation using Pixi	2
1.1	1. What this method installs	2
1.2	2. Install the prerequisites	3
1.3	3. Clone the plugins repository	4
1.4	4. Enter the RunCellpose environment	4
1.5	5. Create and activate the environment	5
1.6	6. Open CellProfiler	6
1.7	7. Configure the plugins folder	6
1.8	8. Verify RunCellpose	7
2	Option 2 — Installation using Docker	8
2.1	9. Install CellProfiler and Docker Desktop	8
2.2	10. Update WSL2	8
2.3	11. Fix virtualization errors	8
2.4	12. Configure CellProfiler	9
2.5	13. Run RunCellpose with Docker	10
3	14. Test the installation	10
4	15. Troubleshooting	11
5	Conclusion	12

This tutorial presents two ways to run CellProfiler with the RunCellpose plugin:

1. Pixi, the recommended method, because it brings CellProfiler, Cellpose, and their dependencies together in a reproducible environment;
2. Docker, a useful alternative for people who prefer to use the CellProfiler Desktop application and avoid working directly with a Python environment in the terminal.

💡 Which method should you choose?

Use Pixi as the first option. This method relies on an environment prepared in the official CellProfiler-plugins repository and reduces compatibility problems between CellProfiler, Cellpose, and their dependencies.

Use Docker if you are not familiar with terminal-based environments, are working on a computer with restrictions on creating local environments, or prefer to use CellProfiler Desktop installed separately.

📘 Official documentation

This material was based on the official instructions on the Using plugins page and on the CellProfiler-plugins repository.

- [Using plugins](#)
- [CellProfiler-plugins repository](#)
- [Supported plugins table](#)

1 Option 1 — Installation using Pixi

1.1 1. What this method installs

The Pixi environment used in this tutorial includes compatible versions of:

- CellProfiler;
- Cellpose;
- RunCellpose;
- Python;
- libraries required to run the plugin.

In this workflow, CellProfiler is launched from within the Pixi environment itself. Therefore, you do not need to install the CellProfiler Desktop application beforehand.

⚠ Environment versions

The environment used as an example is:

```
cellprofiler428_cellpose3
```

This name indicates that it was prepared for CellProfiler 4.2.8 and Cellpose 3. Before starting, check the official repository to see whether a newer Pixi environment or one better suited to your goal is available.

1.2 2. Install the prerequisites

You will need:

- Git;
- Pixi;
- a terminal.

1.2.1 Git for Windows

For this tutorial, we recommend using Git Bash on Windows. It provides a terminal similar to those used on Linux and macOS and was the environment in which this procedure was tested.

Download Git for Windows from:

[Install Git for Windows](#)

Git Bash will be installed together with Git.

i Why do we recommend Git Bash?

The commands in this tutorial worked consistently in Git Bash. Depending on the computer configuration, PowerShell may behave differently because of permissions, execution policies, command interpretation, or PATH configuration.

This does not mean that PowerShell is incompatible, but to reduce variation during training, we recommend running the commands below in Git Bash.

1.2.2 Pixi

Install Pixi by following the official instructions:

Install Pixi

After installation, close and reopen Git Bash.

Check the installation:

```
git --version  
pixi --version
```

1.3 3. Clone the plugins repository

In Git Bash, navigate to a folder where you want to store the repository.

For example:

```
cd folder_name
```

Clone the official repository:

```
git clone https://github.com/CellProfiler/CellProfiler-plugins.git
```

Enter the newly created folder:

```
cd CellProfiler-plugins
```

1.4 4. Enter the RunCellpose environment

Navigate to the Pixi environment folder:

```
cd pixi_envs/cellprofiler428_cellpose3
```

Confirm that the files are present:

```
ls
```

The output should include:

```
pixi.toml  
pixi.lock
```

1.5 5. Create and activate the environment

Run:

```
pixi shell --frozen
```

The `--frozen` option instructs Pixi to use the dependency resolution recorded in `pixi.lock` without updating package versions.

The first installation may take a few minutes.

When the environment is active, the terminal should display something similar to:

```
(cellprofiler_cellpose)
```

Warning about [project]

Pixi may display a message saying that the `[project]` field is deprecated and that `[workspace]` should be used instead.

This warning comes from the `pixi.toml` maintained in the official repository and does not prevent the environment from working. Do not modify the file during this tutorial because it is associated with the existing `pixi.lock`.

Warning about SSL_CERT_DIR

On some Windows computers, Pixi may display a message similar to:

```
WARN ignoring SSL_CERT_DIR
```

If the environment is created normally and the prompt shows `(cellprofiler_cellpose)`, this warning can be ignored.

1.6 6. Open CellProfiler

With the environment active, run:

```
python -m cellprofiler
```

The first startup may take some time.

Whenever you want to open CellProfiler again:

```
cd CellProfiler-plugins/pixi_envs/cellprofiler428_cellpose3
pixi shell --frozen
python -m cellprofiler
```

To leave the Pixi environment after closing CellProfiler:

```
exit
```

1.7 7. Configure the plugins folder

With CellProfiler open:

1. go to CellProfiler → Preferences;
2. locate CellProfiler plugins directory;
3. select the active_plugins folder from the cloned repository;
4. save the preferences;
5. close and reopen CellProfiler.

The path must end with:

```
CellProfiler-plugins/active_plugins
```

Do not select only:

```
CellProfiler-plugins
```

 Error Could not load setup

This error occurs when CellProfiler tries to use the repository root as the plugins folder. Change the setting to:

```
CellProfiler-plugins/active_plugins
```

Then save, close, and reopen CellProfiler.

1.8 8. Verify RunCellpose

After restarting CellProfiler:

1. click Add Modules;
2. search for RunCellpose;
3. add the module to the pipeline.

In the field:

```
Run module in docker or local Python environment
```

select the option corresponding to the local Python environment, not Docker.

 Warnings about other plugins

When CellProfiler starts, messages may appear saying that plugins such as RunImage-JScript, RunOmnipose, or RunStarDist could not be loaded.

This happens because the active_plugins folder contains several plugins, while the cell-profiler428_cellpose3 environment installs only the dependencies required for CellProfiler and RunCellpose with Cellpose 3.

These messages can be ignored when the goal is to use only RunCellpose. The success criterion is that the RunCellpose module appears in the Add Modules panel and completes a test segmentation.

2 Option 2 — Installation using Docker

Docker is an alternative for users who prefer to install CellProfiler Desktop and run Cellpose in a separate container.

2.1 9. Install CellProfiler and Docker Desktop

1. Download CellProfiler:

[CellProfiler releases](#)

2. Download Docker Desktop:

[Docker Desktop](#)

i How does this method work?

CellProfiler Desktop runs normally, while the RunCellpose module uses a Docker image containing Cellpose and its dependencies.

2.2 10. Update WSL2

Docker Desktop uses WSL2 on Windows.

Open PowerShell or Command Prompt as administrator and run:

```
wsl --update
```

When the process is complete:

1. restart the computer;
2. open Docker Desktop;
3. wait until it has fully started.

2.3 11. Fix virtualization errors

If Docker displays:

Docker virtualization support not detected

press Win + R, type:

optionalfeatures.exe

and enable the following options when available:

- Hyper-V;
- Virtual Machine Platform;
- Windows Hypervisor Platform;
- Windows Subsystem for Linux.

Restart the computer.

To check whether virtualization is enabled:

1. open Task Manager with Ctrl + Shift + Esc;
2. go to Performance → CPU;
3. confirm that Virtualization: Enabled is displayed.

If it is disabled, you will need to enable it in the BIOS/UEFI.

2.4 12. Configure CellProfiler

Open CellProfiler Desktop and go to:

File → Preferences

Adjust the following settings:

Setting	Recommendation	Reason
Default Input Folder / Output Folder	Use a local folder with a simple path	Reduces permission problems
Maximum number of workers	Use a value compatible with the available CPU cores	Prevents overloading the computer
Save pipeline and/or file list in addition to project	Select Pipeline	Keeps an independent copy of the pipeline

 Folder organization

Use local folders with simple paths and, when possible, keep them outside synchronization services such as Google Drive or OneDrive.

It is also recommended to use CellProfiler's default input and output folders and avoid the *Elsewhere* option whenever possible.

2.5 13. Run RunCellpose with Docker

1. open Docker Desktop;
2. leave it running in the background;
3. open CellProfiler Desktop;
4. add the RunCellpose module;
5. locate the field:

Run module in docker or local Python environment

6. select Docker.

On the first run, the module should automatically download the appropriate Docker image. This process may take some time, but it occurs only once for that image.

 Do not manually pin an old image unless necessary

Docker image names may change as the plugin and Cellpose versions are updated. Prefer selecting one of the images offered by the current version of the RunCellpose module.

If an older pipeline reports that the image is unavailable, open the module and select one of the currently listed images.

3 14. Test the installation

Build a simple pipeline containing:

Images

NamesAndTypes

Groups

```
RunCellpose
SaveImages
```

Select a test image and run the pipeline.

The installation is working when:

- CellProfiler opens normally;
- RunCellpose appears under Add Modules;
- the module completes segmentation;
- segmented objects are produced without a Cellpose-related error.

4 15. Troubleshooting

Problem or message	Likely cause	What to do
[project] is deprecated	The official environment uses older Pixi syntax	Ignore it and do not modify the manifest during the tutorial
ignoring SSL_CERT_DIR	The certificate variable points to a folder without valid certificates	Ignore it if the environment is created normally
Could not load setup	The plugins folder points to the repository root	Select CellProfiler-plugins/active_plugins
No module named jpype	The RunImageJScript dependency is missing	Ignore it if you will not use this plugin
No module named cellpose_omni	The RunOmnipose dependency is missing	Ignore it if you will not use this plugin
No module named stardist	The RunStarDist dependency is missing	Ignore it if you will not use this plugin
RunCellpose does not appear	Incorrect path or CellProfiler was opened outside the Pixi environment	Confirm active_plugins and open with <code>python -m cellprofiler</code>
Docker does not start	Virtualization or WSL2 is not configured	Update WSL2 and check virtualization

Problem or message	Likely cause	What to do
Cannot connect to Docker daemon	Docker Desktop is not running	Open Docker Desktop before running the pipeline
An older pipeline cannot find the Docker image	The image name has been updated	Select an image available in the current module version

5 Conclusion

For new users, the recommended method is Pixi, because it uses a reproducible environment prepared to bring together compatible versions of CellProfiler and Cellpose.

The everyday Pixi workflow is:

```
cd CellProfiler-plugins/pixi_envs/cellprofiler428_cellpose3
pixi shell --frozen
python -m cellprofiler
```

Docker remains a useful alternative for users who prefer to work with CellProfiler Desktop installed separately or who do not want to manage a Python environment directly.